

HMS Docker Commands Guide

Docker command reference for the SR-AIIMS Hospital Management System project.

Project folder: D:\PROJECTS\HMS_SR

Main Compose file: docker-compose.yml

Production Compose file: docker-compose.prod.yml

Generated on: June 3, 2026

1. What Docker Runs In This HMS

Mode	Services	Ports	Purpose
Main Docker stack	postgres, redis, backend, frontend	5432, 6379, 5000, 5173	Runs the database, cache, Express backend, and nginx-served React frontend together.
Production Docker	backend, frontend	5000, 5173	Builds the Express backend container and nginx frontend container. Frontend proxies /api/v1/ to backend.

Run all commands from the project root: D:\PROJECTS\HMS_SR.

2. Daily HMS Docker Commands

Start the full HMS Docker stack

```
docker compose up -d
```

Starts PostgreSQL, Redis, backend, and frontend from docker-compose.yml. Use this when images already exist and no code rebuild is needed.

Rebuild and start the full HMS Docker stack

```
docker compose up -d --build
```

Builds the backend and frontend images, then starts all four services. Use this after backend/frontend code changes, Dockerfile changes, package changes, or Compose changes.

Check running containers

```
docker compose ps
```

Shows whether postgres, redis, backend, and frontend are running, along with their container names and published ports.

View logs for all development services

```
docker compose logs
```

Shows recent logs for all HMS containers. Useful when checking startup, backend errors, frontend serving, or database/cache connections.

Follow live logs

```
docker compose logs -f
```

Streams logs continuously. Press **Ctrl+C** to stop watching logs; this does not stop the containers.

View only PostgreSQL logs

```
docker compose logs -f postgres
```

Use this when checking database startup, connection failures, migrations, or SQL errors.

View only Redis logs

```
docker compose logs -f redis
```

Use this when checking cache availability or Redis startup issues.

View only backend logs

```
docker compose logs -f backend
```

Use this for Express API errors, database connection issues, login problems, and migration-related problems.

View only frontend logs

```
docker compose logs -f frontend
```

Use this for nginx startup or proxy errors. Normal 200 responses for /login, /patients, and /assets/... mean the React app is being served.

Stop development containers but keep data

```
docker compose down
```

Stops and removes the HMS containers and network. The named volumes postgres_data and redis_data remain, so database data is kept.

Remove old orphan containers, then rebuild

```
docker compose down --remove-orphans  
docker compose up -d --build
```

Use this after Compose service names changed or when old containers such as stale hms_backend or hms_frontend are running outside the current Compose definition.

Stop services without removing containers

```
docker compose stop
```

Stops containers while keeping them available for a quick restart.

Restart stopped development containers

```
docker compose start
```

Starts previously stopped containers without recreating them.

3. Database And Cache Checks

Open PostgreSQL shell inside Docker

```
docker exec -it hms_postgres psql -U hms_user -d hms_db
```

Opens the psql prompt inside the running database container. Use this to inspect tables, run simple SQL checks, or confirm migration results.

List database tables

```
docker exec -it hms_postgres psql -U hms_user -d hms_db -c "\dt"
```

Shows tables in the HMS database without entering the interactive psql shell.

Check PostgreSQL readiness

```
docker exec hms_postgres pg_isready -U hms_user -d hms_db
```

Confirms whether PostgreSQL is ready to accept connections.

Check Redis response

```
docker exec hms_redis redis-cli ping
```

Returns PONG when Redis is running correctly.

4. Backup And Restore Commands

Create a PostgreSQL dump from the Docker container

```
docker exec hms_postgres pg_dump -U hms_user -d hms_db > backups\hms_db_backup.sql
```

Exports the HMS database to a SQL file in the backups folder. Use this before major migrations, imports, or risky cleanup.

Restore a SQL backup into PostgreSQL

```
Get-Content backups\hms_db_backup.sql | docker exec -i hms_postgres psql -U hms_user -d hms_db
```

Restores a SQL dump into the running database container. Use the correct backup filename before running.

The project also includes npm backup helpers: `npm run db:backup` and `npm run db:restore`. Those are not Docker commands, but they are useful for normal HMS database operations.

5. Production Docker Commands

Production Docker uses `.env.production` for the backend. Confirm that file exists on the server before rebuilding containers.

Build and start production containers

```
docker compose -f docker-compose.prod.yml up -d --build
```

Builds hms_backend from backend/Dockerfile, builds hms_frontend from frontend/Dockerfile, and starts both containers in the background.

Start production containers without rebuilding

```
docker compose -f docker-compose.prod.yml up -d
```

Starts containers using existing images. Use this when no code or Dockerfile changes were made.

Check production container status

```
docker compose -f docker-compose.prod.yml ps
```

Shows the backend and frontend production containers, including whether they are running and which ports are exposed.

View production logs

```
docker compose -f docker-compose.prod.yml logs -f
```

Streams logs from both backend and frontend containers. Use this after deployment to catch startup errors.

View backend production logs only

```
docker compose -f docker-compose.prod.yml logs -f backend
```

Use this for Express API errors, database connection issues, authentication failures, and migration-related problems.

View frontend production logs only

```
docker compose -f docker-compose.prod.yml logs -f frontend
```

Use this for nginx startup or proxy errors. The frontend serves on host port 5173 and proxies /api/v1/ to backend:5000.

Restart production containers

```
docker compose -f docker-compose.prod.yml restart
```

Restarts both production containers. Use after changing server-level settings or when containers are stuck but images do not need rebuilding.

Restart only the backend

```
docker compose -f docker-compose.prod.yml restart backend
```

Use after backend-only configuration changes or when the API is unhealthy.

Restart only the frontend

```
docker compose -f docker-compose.prod.yml restart frontend
```

Use when nginx or frontend serving needs a restart without touching the backend container.

Stop production containers but keep images

```
docker compose -f docker-compose.prod.yml down
```

Stops and removes production containers and their Compose network. Built images remain available for the next up.

6. Build And Image Maintenance

Rebuild only backend image

```
docker compose -f docker-compose.prod.yml build backend
```

Builds the backend image after backend code, backend package, or backend Dockerfile changes.

Rebuild only frontend image

```
docker compose -f docker-compose.prod.yml build frontend
```

Builds the frontend image after React code, nginx config, frontend package, or frontend Dockerfile changes.

Rebuild frontend with an API base URL

```
$env:VITE_API_BASE_URL="https://your-domain.example/api/v1"  
docker compose -f docker-compose.prod.yml up -d --build frontend
```

Sets the frontend build-time API base URL for the current PowerShell session, then rebuilds the frontend container. Replace the URL with the real HMS server URL when needed.

List Docker images

```
docker images
```

Shows locally available images, including images built for the HMS backend and frontend.

Remove unused Docker images

```
docker image prune
```

Removes dangling images that are no longer used. Docker asks for confirmation before deleting.

7. Volumes And Data Safety

List Docker volumes

```
docker volume ls
```

Shows named volumes. For local development, watch for `hms_sr_postgres_data` and `hms_sr_redis_data`, or similarly named project volumes.

Inspect PostgreSQL data volume

```
docker volume inspect hms_sr_postgres_data
```

Shows metadata and mount details for the database volume. The exact volume name can vary depending on the Compose project name.

Danger command: `docker compose down -v` stops containers and deletes named volumes. For this HMS project, that can delete local PostgreSQL and Redis data. Run a backup first.

Reset local Docker data only when intentional

```
docker compose down -v
```

Use only when you want a fresh local database/cache and you are sure existing local data is not needed.

8. Troubleshooting Commands

Show all containers, including stopped ones

```
docker ps -a
```

Useful when a container exited immediately and no longer appears in normal status output.

Inspect a container

```
docker inspect hms_backend
```

Shows full container configuration, network details, environment, mounts, and health information.

Open a shell in the backend container

```
docker exec -it hms_backend sh
```

Use to inspect files or environment inside the backend container.

Open a shell in the frontend container

```
docker exec -it hms_frontend sh
```

Use to inspect nginx files or built frontend assets inside the frontend container.

Validate resolved production Compose config

```
docker compose -f docker-compose.prod.yml config
```

Prints the final Compose configuration after variables are resolved. Use this before deployment if environment variables or build args changed.

9. Recommended HMS Workflows

Main Docker startup

Rebuild after changes and verify services

```
docker compose down --remove-orphans
docker compose up -d --build
docker compose config --services
docker compose ps
```

Removes stale orphan containers, rebuilds backend/frontend, verifies the Compose service list, and checks that containers are running.

Production deployment

Rebuild and monitor production containers

```
docker compose -f docker-compose.prod.yml up -d --build
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs -f
```

Builds the backend/frontend containers, checks their status, and watches logs for errors after deployment.

Before risky database work

Backup first, then migrate/import

```
docker exec hms_postgres pg_dump -U hms_user -d hms_db >
backups\hms_db_backup_before_change.sql
npm run db:migrate
```

Create a backup before migrations, old patient imports, or other database-changing tasks.

Prepared for the HMS project in D:\PROJECTS\HMS_SR. Keep real passwords and production environment values out of shared documents.